

Ubuntu・OS基礎 自力課題

M0～M7 Mission Guide

Ubuntu Server 24.04 LTS / 公開Lab v1.0.0
JDU Cyber Security / 2026-09-25

Ubuntu and OS Foundations — 自力課題 Mission Guide

対象: Mission 0～7 環境: Ubuntu Server 24.04 LTS 進め方: 授業回ではなく、学生ごとの進度でMissionを進める。

この文書は、commandを自分で選ぶ課題M0～M7である。完全なcommand手順が必要な場合は、先にGUIDED_PRACTICE.mdのP1～P6を実施する。

PとMは別のresourceを使う。Pを完了しても、対応するMはPASSにならない。推奨順序はP1 → M1 → P2 → M2 → … → P6 → M6 → M7である。M0と統合課題M7には対応するPを設けない。

最初にUbuntuへ接続する

AWS Academy Learner Labのセットアップを完了してから、AWSコンソールでCloudShellを開く。CloudShellで次のcommandを実行し、自分のUbuntu演習サーバーへ接続する。

```
ssh jdu-ubuntu
```

接続後、Ubuntuで次を実行する。

```
id -un
```

ssm-userと表示されることを確認する。この文書のUbuntu側の操作は、この接続先で実行する。CloudShellで操作する課題では、課題中の指示に従ってexitでCloudShellへ戻る。

共通操作

初回構築時に、すべてのMissionは自動で未完成の状態になる。最初にresetする必要はない。

実行user

Ubuntuへ接続した直後の管理userはssm-userである。次のcommandで確認する。

```
id -un
```

jdu-checkとjdu-resetは、必ずssm-userで実行する。別のuserで実行すると、ssm-userへ戻る方法を表示して終了する。

演習用userはjduops、jduviewer、jduwriterである。ssm-userから演習用userへ切り替える例を示す。

```
sudo su - jduops
id
```

元のssm-userへ戻る。

```
exit
id -un
```

この演習で切り替えるuserのうち、Passwordなしでsudoを使えるのはssm-userだけである。例えば、jduwriterからjduviewerへ直接切り替えない。一度exitでssm-userへ戻り、そこからsudo su - jduviewerを実行する。

jduapp、jduweb、jdufinal、jduworkerはservice専用userである。Login shellはnologinであり、su-による対話loginは行わない。必要な確認はssm-userからsudo -u USER COMMANDで1 commandだけ実行する。

現在のMissionを確認する。

```
jdu-check M1
```

教員のprogress serverが設定されている場合、現在の結果は自動送信される。全問PASSする前でも送信される。

RESULTはUbuntu上の課題判定である。REPORTはHTTPS送信の成否である。送信に失敗しても、課題のPASS/FAILは変わらない。自分の匿名server IDは次で確認できる。

```
jdu-progress id
```

通信を一時的に行わず、local判定だけを実行する場合は次を使う。

```
jdu-check M1 --no-submit
```

最初からやり直す場合だけ、そのMissionをresetする。

```
jdu-reset M1
```

reset直後は、すべての課題がFAILになる。これは正常である。LabCheckは状態を修正しない。

M0 Environment and OS

Ubuntu、Linux kernel、PID 1、現在のuser、hostnameを実機から確認する。~/jdu-lab/m0/observation.envへ6項目を記録する。

```
OS_ID=  
OS_VERSION_ID=  
KERNEL_RELEASE=  
PID1_COMM=  
USER_NAME=  
HOST_NAME=
```

値は実機のcommand出力から取得する。推測しない。

```
jdu-check M0
```

M1 Shell, path, file, and text

目的

相対pathと絶対pathを区別する。Directoryとfileを作る。Fileをコピーする。Logから必要な行を取り出す。

初期状態

~/jdu-lab/m1/inboxに素材がある。case01は未完成であり、stagingに不要な.tmp fileがある。

課題

1. 次のdirectory treeを作る。

```
~/jdu-lab/m1/case01/
├── config/
├── logs/
└── notes/
```

1. inbox/config/app.confをcase01/config/app.confへコピーする。
2. inbox/logs/incident.logをcase01/logs/incident.logへコピーする。
3. コピーした2 fileの内容を変更しない。
4. case01の中から.tmp fileをすべて除く。
5. case01以下を、Ubuntuへ接続した管理user ssm-userが所有する状態にする。
6. incident.logからERRORを含む完全な行だけを取り出し、notes/errors.txtへ保存する。行番号を付けない。
7. incident.logの最後の5行を順序を変えず、notes/recent.txtへ保存する。

判定

6課題を判定する。Directory、source file、.tmp、owner、errors.txt、recent.txtがそれぞれ課題に対応する。

```
jdu-check M1
```

確認候補: pwd, ls, tree, mkdir, cp, rm, find, grep, tail, stat

M2 User, group, permission, and sudo

目的

User、primary group、supplementary groupを区別する。共有directoryへgroup accessとsetgidを設定する。

初期状態

jduops、jduviewer、opsは存在する。初期状態では、jduviewerだけがsupplementary group opsに誤って所属している。jduopsはopsに所属していない。/srv/jdu-shareとREADME.txtのowner、group、modeは未完成である。

課題

1. jduopsをsupplementary group opsへ追加する。
2. jduviewerをsupplementary group opsから削除する。
3. /srv/jdu-shareをroot:opsにする。
4. /srv/jdu-shareをmode 2775にする。setgidを使う。world-writableにしない。
5. /srv/jdu-share/README.txtをroot:ops、mode 664にする。
6. ssm-userからsudo su - jduopsで切り替える。共有directoryにfileを作る。新しいfileのgroupがopsになることも確認する。
7. jduviewerがREADME.txtを読めることを確認する。
8. jduviewerが共有directoryにfileを作れないことを確認する。

jduopsでの確認後はexitでssm-userへ戻る。次にsudo su - jduviewerで切り替える。jduviewerでの確認後もexitで戻る。最後にid -unがssm-userであることを確認してからjdu-check M2を実行する。

判定

6課題を判定する。Group membership、directory owner/group、directory mode、sample file、group継承、viewer accessが対応する。

```
jdu-check M2
```

確認候補: id, groups, getent passwd, getent group, stat, namei, chown, chmod, usermod, sudo -u

禁止: chmod 777

M3 Process and package

第8・9章と練習P3の両部分を終えてから取り組む。第10章では、サービスの起動と自動起動を詳しく学ぶ。

目的

Serviceとprocessを対応させる。PIDを指定してsignalを送る。Ubuntu packageを調査し、installする。

初期状態

jdu-m3-process1.service、jdu-m3-process2.service、jdu-m3-process3.serviceが通常のsystemd unitとして動いている。cmatrixは未installである。初期状態を確認するには、次を使う。

```
systemctl list-units --type=service 'jdu-m3-*'
systemctl status jdu-m3-process1.service jdu-m3-process2.service jdu-m3-process3.service
```

課題A: process2だけを停止する

1. jdu-m3-process2.serviceのMain PIDを調べる。
2. psで同じPIDのprocessを確認する。
3. そのPIDだけへTERMを送る。
4. process2が停止し、process1とprocess3が動いていることを確認する。

広い条件のpkill、killall、signal KILLは使わない。

課題B: cmatrixをinstallする

1. Package情報を確認する。
2. aptでcmatrixをinstallする。
3. Commandを実行する。終了はCtrl+Cとする。
4. Packageのversionと、/usr/bin/cmatrixを提供するpackageを確認する。

判定

2課題を判定する。課題A全体で1件、課題B全体で1件である。提出fileはない。

```
jdu-check M3
```

確認候補: `systemctl status`, `systemctl show`, `ps`, `kill`, `apt show`, `apt install`, `dpkg-query`, `command -v`, `dpkg -S`

M4 systemd service

目的

Unit file、service、processの関係を確認する。Activeとenabledを区別する。

初期状態

教師が用意した `jdu-status.service` は loaded、inactive、disabled である。Unit file は完成済みである。

課題

1. `systemctl cat jdu-status.service` で `User`、`WorkingDirectory`、`ExecStart` を読む。
2. Unit file を変更せず、service を start する。
3. Boot 時に start するよう enable する。
4. Active と enabled を別々に確認する。
5. Main PID を調べ、実際の process と対応させる。
6. Process の user、command line、working directory が unit の指定と一致することを確認する。

判定

3課題を判定する。Active、enabled、変更されていない unit と実 process の対応がそれぞれ課題になる。提出 file はない。

```
jdu-check M4
```

確認候補: `systemctl cat`, `systemctl start`, `systemctl enable`, `systemctl is-active`, `systemctl is-enabled`, `systemctl show`, `ps`, `/proc/PID/cmdline`, `/proc/PID/cwd`

M5 Port, socket, and log

目的

Service、process、socket、IP address、port、HTTP、journal を一つの実行状態として関連付ける。

初期状態

教師が用意した `jdu-web.service` は loaded、inactive、disabled である。Unit file と content file は完成済みである。TCP 8081 には listener がいない。

課題

1. Unit file の `User`、`ExecStart`、`address`、`port`、`content path` を読む。
2. Unit file を変更せず、`jdu-web.service` を start する。
3. `127.0.0.1:8081` で通信を待ち受けている socket を確認する。この状態を listener (待受 socket) という。

```
sudo ss -ltnp | grep ':8081'
```

1. `ss` の `users:` 欄にある PID と、service の Main PID を比較する。

```
systemctl show --property MainPID --value jdu-web.service
```

1. 0.0.0.0:8081や[::]:8081でlistenしていないことを確認する。
2. http://127.0.0.1:8081/へrequestを送り、HTTP responseを確認する。
3. Service user jduwebがcontent fileを読めることを確認する。jduwebはlogin用userではないため、su - jduwebは使わない。実際のread可否は次で確認する。0はread可能、1はread不可を表す。

```
sudo -u jduweb -- test -r /srv/jdu-web/index.txt
echo $?
```

Userとgroupはgetent passwd jduwebとid jduwebで確認する。Fileと親directoryのpermissionはstatとnamei -l /srv/jdu-web/index.txtで確認する。

1. http://127.0.0.1:8081/m5-checkへrequestを送る。
2. 現在のservice起動に対応するjournalでREQUEST path=/m5-checkを確認する。
3. TCP 18081にlistenerがなく、HTTP requestが失敗することを確認する。

判定

4課題を判定する。ListenerとPID、HTTPとfile access、学生が送ったrequestのlog、open portとclosed portの比較が対応する。LabCheckが送る/requestは、学生の/m5-checkを代行しない。

```
jdu-check M5
```

確認候補: systemctl cat, systemctl start, systemctl show, sudo ss -ltnp, curl, sudo -u, sudo journalctl

M6 SSH and remote operation

目的

CloudShellをlocal、Ubuntuをremoteとして区別する。SSH remote commandとscpのupload、downloadを使う。

初期状態

SSH鍵、alias jdu-ubuntu、Session Manager tunnelは構築済みである。これらは課題点に含めない。CloudShellとUbuntuのMission fileは存在しない。

課題

1. CloudShellでid -un、hostname、pwdを確認する。
2. ssh jdu-ubuntuまたはSSH remote commandで、Ubuntu側の同じ3項目を確認する。
3. CloudShellの~/jdu-lab/m6/local-source.txtへ、次の一行だけを書く。

```
JDU SSH transfer test
```

1. scpで、Ubuntuの~/jdu-lab/m6/upload.txtへuploadする。
2. SSH remote commandを使い、Ubuntuに~/jdu-lab/m6/remote-result.txtを作る。値を手入力しない。

```
REMOTE_USER=<Ubuntuで取得したuser>
REMOTE_HOST=<Ubuntuで取得したhostname>
REMOTE_PATH=<Ubuntuで取得したhome directory>
```

1. scpで、Ubuntuのremote-result.txtをCloudShellの~/jdu-lab/m6/downloaded-result.txtへdownloadする。
2. Upload元と先、download元と先をsha256sumで比較する。

判定

Ubuntu側は2課題を判定する。UbuntuへSSHで入った状態で実行する。この結果はDashboardのM6 Ubuntuへ送られる。

```
jdu-check M6
```

CloudShell側は4課題を判定する。UbuntuからexitしてCloudShellへ戻った後に実行する。この結果はDashboardのM6 CloudShellへ送られる。

```
jdu-check M6
```

DashboardのM6欄は、Ubuntu 2/2、CloudShell 4/4、合計6/6を別々に表示する。M6のMission完了は両方が全件PASSになった時点である。どちらか一方だけを再実行しても、もう一方の最新結果は保持される。

両方で全件PASSにする。秘密鍵を表示、提出、移動しない。SSH service、SSH設定、authorized_keysを変更しない。

確認候補: ssh, scp, sha256sum, id, hostname, pwd, printf

M7 Integrated Ubuntu check

目的

M2、M4、M5の知識を、新しいserviceへ自力で適用する。新しいcommandは追加しない。

初期状態

- 教師配布のjdu-final.serviceはloaded、inactive、disabledである。
- /srv/jdu-finalとindex.txtは存在しない。
- jduwriterとjduviewerはfinalopsに所属していない。

課題

1. systemctl cat jdu-final.serviceでUser、WorkingDirectory、ExecStartを読む。
2. /srv/jdu-finalをroot:finalops、mode 2775で作る。
3. /srv/jdu-final/index.txtを作る。内容は/etc/jdu-lab/final-markerと同じ一行にする。
4. index.txtをroot:finalops、mode 664にする。
5. jduwriterをfinalopsへ追加する。jduviewerは追加しない。
6. Writerがcontentを更新でき、viewerが更新できないことを確認する。
7. Unit fileを変更せず、serviceをstartしてenableする。
8. Serviceがjdufinalで動き、127.0.0.1:8090だけでlistenすることを確認する。

9. HTTP responseがstatus 200で、指定markerを返すことを確認する。

10. `http://127.0.0.1:8090/m7-check`へrequestを送る。

11. 現在のservice起動に対応するjournalでREQUEST path=/m7-checkを確認する。

Userを切り替える場合は、`ssm-user`から`sudo su - jduwriter`または`sudo su - jduviewer`を使う。各確認後は`exit`で`ssm-user`へ戻る。最後に`id -un`を確認してから`jdu-check M7`を実行する。

判定

6課題を判定する。File構成、writer/viewer、service、socket、HTTP、journalが対応する。Unit file、SSH設定、LabCheckを変更しない。

```
jdu-check M7
```

確認候補: `id`, `getent`, `stat`, `chown`, `chmod`, `systemctl`, `ps`, `sudo ss -lntp`, `curl`, `sudo journalctl`

安全上の禁止事項

- `chmod 777`を使わない。
- `/etc/ssh/sshd_config`を変更しない。
- `ssh.service`を停止しない。
- `killall`または広い条件の`pkill`を使わない。
- 秘密鍵を表示、提出、Gitへ追加しない。
- 外部hostへscanまたはlogin試行をしない。